

EX.NO: 01

DATE:

STORED PROCEDURE

Aim:

To create a stored procedure and displays the details of an employee based on the employee ID.

Procedure:

Step 1: To Create the employees table

```
CREATE TABLE employees (  
    emp_id NUMBER PRIMARY KEY,  
    emp_name VARCHAR2(100),  
    salary NUMBER(10, 2)  
);
```

Step 2: Insert sample data

```
INSERT INTO employees VALUES (1, 'Alice', 5000);  
INSERT INTO employees VALUES (2, 'Bob', 6000);  
INSERT INTO employees VALUES (3, 'Charlie', 7000);
```

Step 3: Create the stored procedure

```
CREATE OR REPLACE PROCEDURE GetEmployeeDetails (  
    p_emp_id IN NUMBER  
)  
IS  
BEGIN  
    FOR rec IN (  
        SELECT emp_id, emp_name, salary  
        FROM employees  
        WHERE emp_id = p_emp_id  
    ) LOOP  
        DBMS_OUTPUT.PUT_LINE('ID: ' || rec.emp_id);  
        DBMS_OUTPUT.PUT_LINE('Name: ' || rec.emp_name);  
        DBMS_OUTPUT.PUT_LINE('Salary: ' || rec.salary);  
    END LOOP;  
END;  
/
```

Step 4: Enable server output and execute the procedure

```
BEGIN  
    GetEmployeeDetails(2);  
END;  
/
```

Output:



ID: 2

Name: Bob

Salary: 6000

Result:

The stored procedure was created and executed successfully. It retrieved and displayed the employee details for the given ID as expected.

RECURSIVE PROCEDURE

Aim:

To create and execute a recursive stored procedure that demonstrating recursion on hierarchical table data.

Procedure:**Step 1:** To Create the employees table

```
CREATE TABLE employees (  
    emp_id NUMBER PRIMARY KEY,  
    emp_name VARCHAR2(50),  
    manager_id NUMBER  
);
```

Step 2: Insert sample hierarchical data:

```
INSERT ALL  
    INTO employees (emp_id, emp_name, manager_id) VALUES (1, 'Alice', NULL)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (2, 'Bob', 1)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (3, 'Charlie', 1)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (4, 'David', 2)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (5, 'Eve', 2)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (6, 'Frank', 3)  
SELECT * FROM dual;
```

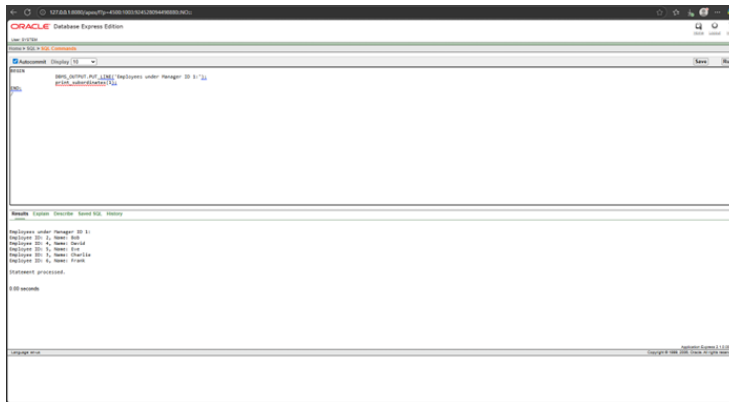
Step 3: Create the recursive procedure to print subordinates:

```
CREATE OR REPLACE PROCEDURE print_subordinates (p_manager_id IN NUMBER) AS  
BEGIN  
    FOR emp IN (  
        SELECT emp_id, emp_name FROM employees WHERE manager_id = p_manager_id  
    ) LOOP  
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp.emp_id || ', Name: ' || emp.emp_name);  
        print_subordinates(emp.emp_id);  
    END LOOP;  
END;  
/
```

Step 4: Execute the procedure for a manager

```
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Employees under Manager ID 1:');  
    print_subordinates(1);  
END;  
/
```

Output:



Employees under Manager ID 1:

Employee ID: 2, Name: Bob

Employee ID: 4, Name: David

Employee ID: 5, Name: Eve

Employee ID: 3, Name: Charlie

Employee ID: 6, Name: Frank

Result:

The stored procedure executed successfully and displayed all employees reporting to the given manager as expected.

CURSORS

Aim:

To create and use cursor to fetch and display employee data from an hierarchical table.

Procedure:**Step 1:** To Create the employees table

```
CREATE TABLE employees (  
    emp_id NUMBER PRIMARY KEY,  
    emp_name VARCHAR2(50),  
    manager_id NUMBER  
);
```

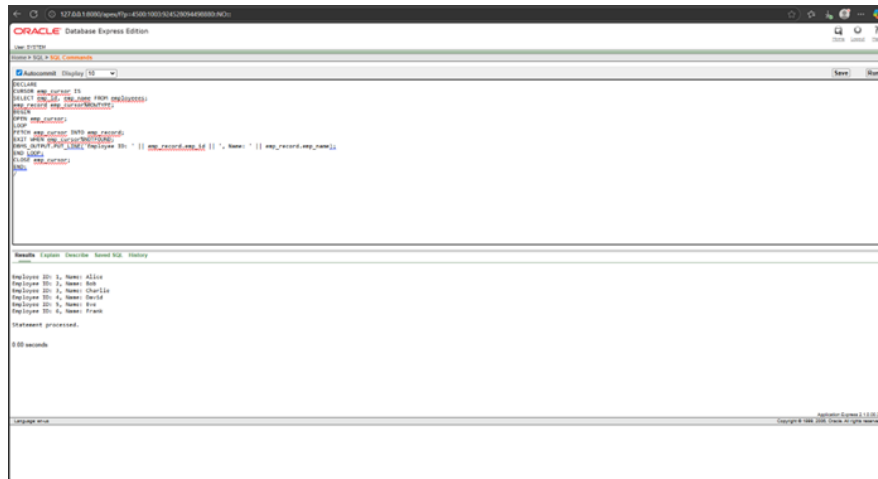
Step 2: Insert sample data:

```
INSERT ALL  
    INTO employees (emp_id, emp_name, manager_id) VALUES (1, 'Alice', NULL)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (2, 'Bob', 1)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (3, 'Charlie', 1)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (4, 'David', 2)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (5, 'Eve', 2)  
    INTO employees (emp_id, emp_name, manager_id) VALUES (6, 'Frank', 3)  
SELECT * FROM dual;
```

Step 3: Create and use an explicit cursor to fetch employee data:

```
DECLARE  
    CURSOR emp_cursor IS  
        SELECT emp_id, emp_name FROM employees;  
    emp_record emp_cursor%ROWTYPE;  
BEGIN  
    OPEN emp_cursor;  
    LOOP  
        FETCH emp_cursor INTO emp_record;  
        EXIT WHEN emp_cursor%NOTFOUND;  
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.emp_id || ', Name: ' || emp_record.emp_name);  
    END LOOP;  
    CLOSE emp_cursor;  
END;  
/
```

Output:



The screenshot shows the Oracle Database Express Edition interface. The main window displays a PL/SQL script that defines a cursor to fetch employee IDs and names from the 'employees' table. The script includes a loop to process each row. Below the script, the 'Results' pane shows the output of the script, which is a list of employee IDs and names. The output is as follows:

Employee ID	Name
1	Alice
2	Bob
3	Charlie
4	David
5	Eve
6	Frank

Employee ID: 1, Name: Alice
Employee ID: 2, Name: Bob
Employee ID: 3, Name: Charlie
Employee ID: 4, Name: David
Employee ID: 5, Name: Eve
Employee ID: 6, Name: Frank

Result:

The cursor successfully fetched and displayed employee IDs and names from the employees table.

USER MANAGEMENT

Aim:

The objective of this lab exercise is to create new users , modify and delete the users and their privilege.

Procedure:

Step 1: User Creation

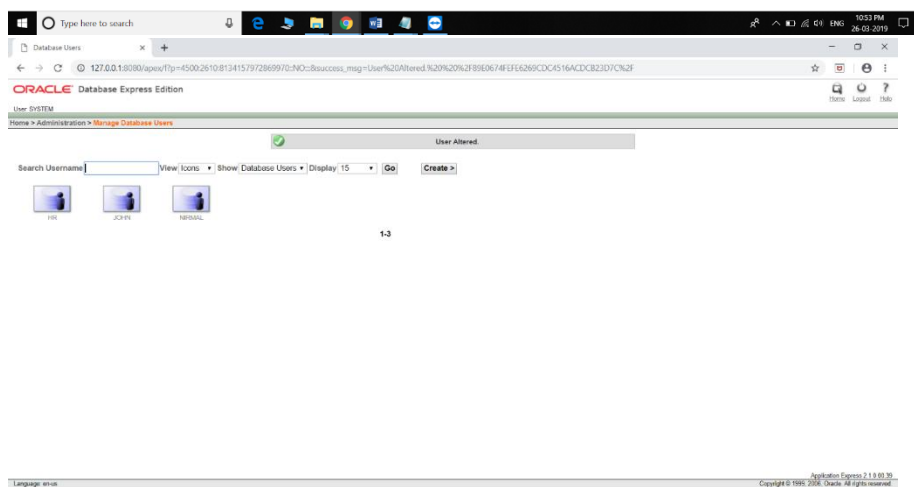
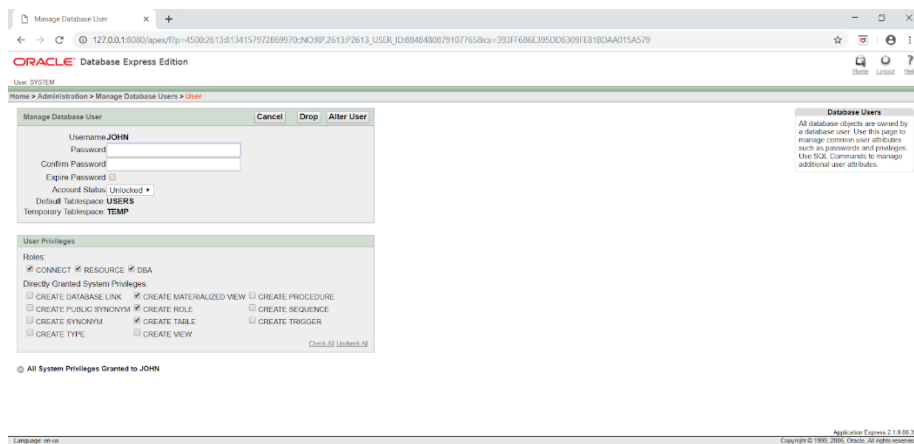
1. click the Administration icon and then Database Users.
2. Enter the appropriate administrator username and password and click Login
3. The Manage Database Users page appears. Click Create button.
4. The Create Database User page appears. Fields marked with a red asterisk (*) are mandatory required.
5. Username – Enter a new username for database authentication.
6. Password – Enter a password for database authentication.
7. Confirm Password – Enter the same password again.
8. Select Expire Password to prompt users to change their password the next time they log in.
9. Use Account Status to control whether a user can log into the account.
Valid options include:
 - i. Unlocked. Select this option to enable a user to log in.
 - ii. Locked. Select this option to prevent a user from logging in.
10. Select the roles and privilege according to the user profile needs.
11. Click Create button.
12. To edit account details, select a user.
13. The User Page appears.
14. Add any ROLE in the database and click on Alter user.
15. The database user will be altered.

Step 2: Delete users

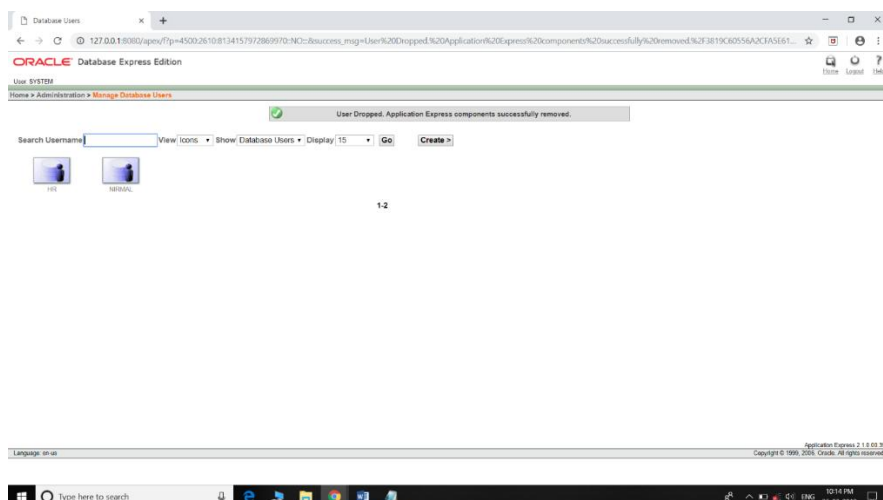
1. Select the user account.
2. click on drop option.
3. Again, click on the drop user option to remove or delete the account.
4. To view the list of logins select show from manage database users.
5. Click on all users and select go.
6. The list of users will be displayed.

Output:

User creation and modification:



Deletion of user:



Result:

Database users were successfully created, modified, and deleted. User roles and privileges were managed as required.

EX.NO: 05

DATE:

INDEX MANAGEMENT

Aim:

To create, manage, and use indexes to improve query performance.

Procedure:

Step-1: Create a Sample Table

```
CREATE TABLE Employees (  
    EmpID NUMBER PRIMARY KEY,  
    Name VARCHAR2(50),  
    Department VARCHAR2(50),  
    Salary NUMBER  
);
```

Step-2: Insert Sample Data

```
INSERT INTO Employees VALUES (1, 'Alice', 'HR', 5000);  
INSERT INTO Employees VALUES (2, 'Bob', 'IT', 7000);  
INSERT INTO Employees VALUES (3, 'Charlie', 'Finance', 6000);  
INSERT INTO Employees VALUES (4, 'David', 'IT', 6500);  
INSERT INTO Employees VALUES (5, 'Eva', 'HR', 5200);
```

Step-3: Create a non-unique index

```
CREATE INDEX idx_department ON Employees (Department);
```

Step-4: Create a composite index:

```
CREATE INDEX idx_dept_salary ON Employees (Department, Salary);
```

Step-5: Verify Index Creation

```
SELECT index_name, table_name, uniqueness FROM user_indexes WHERE table_name =  
'EMPLOYEES';
```

Verified that the two indexes are created successfully

Step-6: To view the execution plan

```
EXPLAIN PLAN FOR  
SELECT * FROM Employees WHERE Department = 'IT'
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY());
```

Step-7: Drop an Index

```
DROP INDEX idx_department;
```

Output:

Home > SQL > SQL Commands

Autocommit Display 10

```
SELECT index_name, table_name, uniqueness FROM user_indexes WHERE table_name = 'EMPLOYEES';
```

Results Explain Describe Saved SQL History

INDEX_NAME	TABLE_NAME	UNIQUENESS
SYS_C004003	EMPLOYEES	UNIQUE
IDX_DEPARTMENT	EMPLOYEES	NONUNIQUE
IDX_DEPT_SALARY	EMPLOYEES	NONUNIQUE

3 rows returned in 0.00 seconds [CSV Export](#)

ORACLE Database Express Edition

Home > SQL > SQL Commands

Autocommit Display 10

```
SELECT * FROM TABLE(DBMS_XPUL.GETSQL(1));
```

Details Explain Describe Saved SQL History

PLAN_TABLE_OUTPUT

Plan hash value: 14542707

(1) Operation Name Rows Bytes Cost (CPU% Mem%)

(1) SELECT STATEMENT (1) 403 2.8K 59.50 (91)

(1) TABLE ACCESS FULL EMPLOYEES (1) 362 2.8K 46.40 (91)

Predicate Information Identified by operation(s):

More than 10 rows available. Increase row selector to view more rows

10 rows returned in 0.06 seconds [SQL Trace](#)

Language error

Oracle Database Express Edition 11.2.0.3.0
Copyright (c) 2009, 2010, Oracle. All rights reserved.

Result:

Successfully Created and managed indexes on *Employees* to improve query performance.

EX.NO: 06

DATE:

MATERIALIZED VIEW

Aim:

To create a materialized view in Oracle Database XE and understand how to refresh and query it.

Procedure:

Step 1: To Create table:

```
CREATE TABLE orders (  
    order_id NUMBER PRIMARY KEY,  
    customer_id NUMBER,  
    order_date DATE,  
    amount NUMBER(10,2)  
);
```

Step 2: Insert sample data:

```
INSERT INTO orders VALUES (1, 101, TO_DATE('2025-07-01', 'YYYY-MM-DD'), 500);  
INSERT INTO orders VALUES (2, 102, TO_DATE('2025-07-03', 'YYYY-MM-DD'), 700);  
INSERT INTO orders VALUES (3, 101, TO_DATE('2025-07-05', 'YYYY-MM-DD'), 200);  
INSERT INTO orders VALUES (4, 103, TO_DATE('2025-07-07', 'YYYY-MM-DD'), 1000);
```

Step 3: Create the materialized view:

```
CREATE MATERIALIZED VIEW sales_summary_mv  
BUILD IMMEDIATE  
REFRESH COMPLETE ON DEMAND  
AS  
SELECT customer_id, SUM(amount) AS total_sales  
FROM orders  
GROUP BY customer_id;
```

- BUILD IMMEDIATE means the materialized view is populated immediately.
- REFRESH COMPLETE ON DEMAND means the data refresh is manual and complete.

Step 4: Query the materialized view:

```
SELECT * FROM sales_summary_mv;
```

Output:



The screenshot shows the Oracle Database Express Edition web interface. The top navigation bar includes the Oracle logo and 'Database Express Edition'. Below the navigation bar, there's a 'SQL Commands' tab selected. The main area contains a SQL query: `SELECT * FROM sales_summary_mv;`. Below the query, the 'Results' tab is active, displaying a table with two columns: 'CUSTOMER_ID' and 'TOTAL_SALES'. The table contains three rows of data. Below the table, it states '3 rows returned in 0.00 seconds'. The bottom of the interface shows the 'Language: en-us' and 'Copyright © 1999, 2009, Oracle. All rights reserved.' text.

CUSTOMER_ID	TOTAL_SALES
102	700
101	700
103	1000

Result:

Materialized view was successfully created and data was viewed. It provides faster access to the summarized data after refreshing.

EX.NO: 07

DATE:

TABLE SPACE MANAGEMENT

Aim:

To understand and perform basic tablespace management operations in Oracle Database Express Edition.

Procedure:

Step 1: Check existing tablespaces

```
SELECT tablespace_name, status, contents FROM dba_tablespaces;
```

Step 2: Create a new tablespace

```
CREATE TABLESPACE my_tablespace  
DATAFILE 'C:\oracle\oradata\XE\my_tablespace.dbf'  
SIZE 50M  
AUTOEXTEND ON  
NEXT 10M MAXSIZE 200M;
```

Step 3: Create a user and assign tablespace

```
CREATE USER test_user IDENTIFIED BY password  
DEFAULT TABLESPACE my_tablespace  
QUOTA UNLIMITED ON my_tablespace;
```

```
GRANT CONNECT, RESOURCE TO test_user;
```

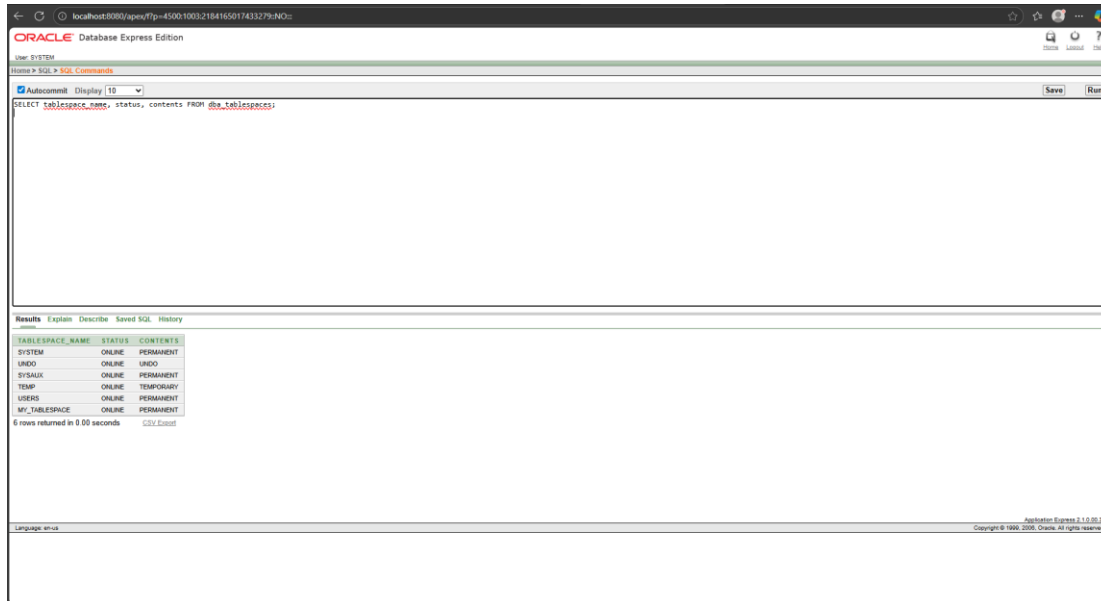
Step 4: Check tablespace usage

```
SELECT tablespace_name, file_name, bytes/1024/1024 AS size_mb FROM dba_data_files;
```

Step 5: Check existing tablespace

```
SELECT tablespace_name, status, contents FROM dba_tablespaces;
```

Output:



Result:

Tablespace was successfully created and assigned to the user. Tablespace details were viewed and space is managed properly.